

```

%% 문제 1
clear; clc;

rng(14);
request = poissrnd(18, 30, 12);
dailyTotal = sum(request, 2);
busyDay = dailyTotal >= 240;
numBusyDay = sum(busyDay);
hourlyMean = mean(request, 1);

figure;
plot(1:30, dailyTotal, '-o');
hold on;
plot(find(busyDay), dailyTotal(busyDay), 'p', 'MarkerSize', 12);
xlabel('Day');
ylabel('Total Requests');
title('Daily Delivery Request Simulation');
grid on;

%% 문제 2
clear; clc;

n = 3000;
interArrival = exprnd(1.2, 1, n);
arrival = cumsum(interArrival);
service = exprnd(0.9, 1, n);

startTime = zeros(1, n);
finishTime = zeros(1, n);

startTime(1) = arrival(1);
finishTime(1) = startTime(1) + service(1);

for i = 2:n
    startTime(i) = max(arrival(i), finishTime(i-1));
    finishTime(i) = startTime(i) + service(i);
end

waitingTime = startTime - arrival;

meanWait = mean(waitingTime);
maxWait = max(waitingTime);
longWaitRatio = sum(waitingTime >= 5) / n;

figure;
histogram(waitingTime, 40);
title('Waiting Time Distribution in a Data Center Queue');
xlabel('Waiting Time');
ylabel('Frequency');
grid on;

```

%%문제 3

```
clear; clc;
```

```
n = 100000;
```

```
c1 = rand(1, n) < 0.97;
```

```
c2 = rand(1, n) < 0.95;
```

```
c3 = rand(1, n) < 0.96;
```

```
c4 = rand(1, n) < 0.92;
```

```
systemWorking = c1 & c2 & c3 & c4;
```

```
reliabilityEst = mean(systemWorking);
```

```
reliabilityTrue = 0.97 * 0.95 * 0.96 * 0.92;
```

```
absError = abs(reliabilityEst - reliabilityTrue);
```

```
failCounts = [sum(~c1), sum(~c2), sum(~c3), sum(~c4), sum(~systemWorking)];
```

```
figure;
```

```
bar(failCounts);
```

```
xticklabels({'C1', 'C2', 'C3', 'C4', 'System'});
```

```
xlabel('Component');
```

```
ylabel('Failure Count');
```

```
title('Failure Counts of Components and System');
```

```
grid on;
```

%% 문제 4

```
clear; clc;
```

```
batteryLife = 9.5 + 1.2 * randn(1, 50000);
```

```
lowBattery = batteryLife < 8;
```

```
normalBattery = batteryLife >= 8 & batteryLife < 11;
```

```
excellentBattery = batteryLife >= 11;
```

```
gradeCounts = [sum(lowBattery), sum(normalBattery), sum(excellentBattery)];
```

```
gradeRatio = gradeCounts / numel(batteryLife);
```

```
figure;
```

```
histogram(batteryLife, 50);
```

```
hold on;
```

```
xline(8);
```

```
xline(11);
```

```
xlabel('Battery Life');
```

```
ylabel('Frequency');
```

```
title('Battery Life Distribution');
```

```
grid on;
```

```
figure;
```

```

bar(gradeCounts);
xticklabels({'Low', 'Normal', 'Excellent'});
xlabel('Grade');
ylabel('Count');
title('Battery Grade Counts');
grid on;

%% 문제 5
clear; clc;

n = 20000;

hasClaim = rand(1, n) < 0.06;
claimAmount = 800 + 250 * randn(1, n);
claimAmount(claimAmount < 0) = 0;

claimPayment = hasClaim .* claimAmount;

totalPayment = sum(claimPayment);
meanPayment = mean(claimPayment);
meanClaimOnly = mean(claimPayment(claimPayment > 0));

numLargeClaim = sum(claimPayment >= 1200);

figure;
histogram(claimPayment);
xlabel('Claim Payment');
ylabel('Frequency');
title('Histogram of All Claim Payments');
grid on;

figure;
histogram(claimPayment(claimPayment > 0));
xlabel('Claim Payment');
ylabel('Frequency');
title('Histogram of Positive Claim Payments');
grid on;

%% 문제 6
clear; clc;

n = 1000;
m = 100;

steps = 2 * (rand(n, m) < 0.56) - 1;
walks = cumsum(steps, 1);

figure;
plot(1:n, walks);
xlabel('Step');

```

```

ylabel('Position');
title('Random Walk Paths');
grid on;

finalPosition = walks(end, :);

meanFinal = mean(finalPosition);
stdFinal = std(finalPosition);
rightDriftRatio = sum(finalPosition >= 100) / m;

figure;
histogram(finalPosition, 30);
xlabel('Final Position');
ylabel('Frequency');
title('Distribution of Final Positions');
grid on;

%% 문제 7
clear; clc;

rng(7);
t = linspace(0, 30, 300);

trueSignal = 2 * sin(0.4 * t) + 0.7 * cos(1.5 * t);
obsSignal = trueSignal + 0.25 * randn(size(t));

missingIdx = randperm(numel(t), 45);
obsSignal(missingIdx) = NaN;

valid = ~isnan(obsSignal);

recLinear = interp1(t(valid), obsSignal(valid), t, 'linear');
recSpline = interp1(t(valid), obsSignal(valid), t, 'spline');

rmseLinear = sqrt(mean((recLinear - trueSignal).^2));
rmseSpline = sqrt(mean((recSpline - trueSignal).^2));

figure;
plot(t, trueSignal, 'LineWidth', 1.2);
hold on;
plot(t, obsSignal, 'o');
plot(t, recLinear, 'LineWidth', 1.2);
plot(t, recSpline, 'LineWidth', 1.2);
legend('True Signal', 'Observed Signal with Missing Data', 'Linear Interpolation', 'Spline Interpolation');
xlabel('Time');
ylabel('Signal');
title('Signal Reconstruction with Interpolation');
grid on;

```

```

%% 문제 8
clear; clc;

timeObs = [0 3 6 9 12 15 18 21 24];
windObs = [3.1 3.8 5.4 7.2 8.0 7.1 5.6 4.2 3.5];

timeHourly = 0:1:24;

windLinear = interp1(timeObs, windObs, timeHourly, 'linear');
windSpline = interp1(timeObs, windObs, timeHourly, 'spline');

diffWind = windSpline - windLinear;
maxDiff = max(abs(diffWind));

figure;
plot(timeObs, windObs, 'o');
hold on;
plot(timeHourly, windLinear, '-');
plot(timeHourly, windSpline, '--');
legend('Observed Data', 'Linear Interpolation', 'Spline Interpolation');
xlabel('Time');
ylabel('Wind Speed');
title('Wind Speed Interpolation');
grid on;

figure;
plot(timeHourly, diffWind, '-o');
xlabel('Time');
ylabel('Difference');
title('Difference between Spline and Linear Interpolation');
grid on;

%% 문제 9
clear; clc;

[X, Y] = meshgrid(1:5, 1:5);

H = [12 14 16 15 13;
     13 18 22 20 16;
     15 23 31 25 18;
     14 20 26 24 17;
     11 15 18 16 14];

[Xq, Yq] = meshgrid(1:0.1:5, 1:0.1:5);

HqLinear = interp2(X, Y, H, Xq, Yq, 'linear');
HqSpline = interp2(X, Y, H, Xq, Yq, 'spline');

figure;
tiledlayout(1, 3);

```

```

nexttile;
surf(X, Y, H);
shading interp;
colorbar;
xlabel('X'); ylabel('Y'); zlabel('Height');
title('Original Terrain');

nexttile;
surf(Xq, Yq, HqLinear);
shading interp;
colorbar;
xlabel('X'); ylabel('Y'); zlabel('Height');
title('Linear Interpolation');

nexttile;
surf(Xq, Yq, HqSpline);
shading interp;
colorbar;
xlabel('X'); ylabel('Y'); zlabel('Height');
title('Spline Interpolation');

%% 문제 10
clear; clc;

tObs = [0 1.2 2.8 4.5 7.0 8.1 9.7 12.0];
cObs = [5.0 6.4 9.1 8.6 6.8 5.9 5.2 4.8];

tq = 0:0.1:12;

cqLinear = interp1(tObs, cObs, tq, 'linear');
cqSpline = interp1(tObs, cObs, tq, 'spline');

deltaC = diff(cqLinear);
[~, idxMaxChange] = max(abs(deltaC));

changeInterval = [tq(idxMaxChange), tq(idxMaxChange + 1)];

figure;
plot(tObs, cObs, 'o');
hold on;
plot(tq, cqLinear, '-');
plot(tq, cqSpline, '--');
plot(changeInterval, cqLinear(idxMaxChange:idxMaxChange+1), 's', 'MarkerSize', 10);
legend('Observed Data', 'Linear Interpolation', 'Spline Interpolation', 'Maximum Change Interval');
xlabel('Time');
ylabel('Concentration');
title('Concentration Interpolation and Maximum Change Interval');
grid on;

```

```

%% 문제 11
clear; clc;

t = linspace(0, 80, 12000);
y = exp(-0.04 * t) .* sin(2 * t) + 0.3 * exp(-0.01 * t) .* cos(7 * t);

figure('Color', 'w');
plot(t, y, 'LineWidth', 1.3);
yline(0);
xlabel('Time');
ylabel('Signal Amplitude');
title('High-Resolution Damped Oscillatory Signal');
xlim([0 80]);
grid on;
box on;
set(gca, 'FontSize', 12, 'LineWidth', 1.2);

print(gcf, 'damped signal.png', '-dpng', '-r300');

%% 문제 12
clear; clc;

x = linspace(-5, 5, 600);
y = linspace(-5, 5, 600);

[X, Y] = meshgrid(x, y);

Z = exp(-0.06 * (X.^2 + Y.^2)) .* sin(4 * X) .* cos(3 * Y) + 0.2 * sin(X .* Y);

figure;
surf(X, Y, Z);
shading interp;
colormap(jet);
colorbar;
xlabel('X');
ylabel('Y');
zlabel('Z');
title('Detailed Surface Plot with Oscillation and Damping');
view(45, 30);
camlight headlight;
lighting phong;

maxZ = max(Z(:));
minZ = min(Z(:));

[~, idxMax] = max(Z(:));
[~, idxMin] = min(Z(:));

[rowMax, colMax] = ind2sub(size(Z), idxMax);

```

```

[rowMin, colMin] = ind2sub(size(Z), idxMin);

maxPoint = [X(rowMax, colMax), Y(rowMax, colMax), Z(rowMax, colMax)];
minPoint = [X(rowMin, colMin), Y(rowMin, colMin), Z(rowMin, colMin)];

%% 문제 13
clear; clc;

x = linspace(-8, 8, 800);
y = linspace(-8, 8, 800);

[X, Y] = meshgrid(x, y);

Z = (cos(2 * X) .* sin(3 * Y)) ./ (1 + 0.1 * (X.^2 + Y.^2));

figure;
contourf(X, Y, Z, 70, 'LineColor', 'none');
colormap(parula);
colorbar;
axis equal;
grid on;
box on;
xlabel('X coordinate');
ylabel('Y coordinate');
title('High-Resolution Filled Contour of a Damped Oscillatory Field');
set(gca, 'FontSize', 12, 'LineWidth', 1.2);

%% 문제 14
clear; clc;

x = linspace(-4, 4, 300);
y = linspace(-4, 4, 300);

[X, Y] = meshgrid(x, y);

Z1 = sin(X) .* cos(Y);
Z2 = exp(-0.08 * (X.^2 + Y.^2));
Z3 = sin(X.^2 + Y.^2);

figure;
tiledlayout(1, 3);

nexttile;
surf(X, Y, Z1);
shading interp;
colorbar;
xlabel('X'); ylabel('Y'); zlabel('Z');
title('Z1 = sin(X)cos(Y)');
view(45, 30);

```



```

nexttile;
surf(X, Y, Z2);
shading interp;
colorbar;
xlabel('X'); ylabel('Y'); zlabel('Z');
title('Z2 = exp(-0.08(X^2+Y^2))');
view(45, 30);

nexttile;
surf(X, Y, Z3);
shading interp;
colorbar;
xlabel('X'); ylabel('Y'); zlabel('Z');
title('Z3 = sin(X^2+Y^2)');
view(45, 30);

sgtitle('Comparison of Three Surface Structures');

%% 문제 15
clear; clc;

u = linspace(0, 8*pi, 500);
v = linspace(-pi, pi, 180);

[U, V] = meshgrid(u, v);

X = (2 + 0.4 * cos(V)) .* cos(U);
Y = (2 + 0.4 * cos(V)) .* sin(U);
Z = 0.25 * U + 0.4 * sin(V);

figure;
surf(X, Y, Z);
shading interp;
colormap(parula);
colorbar;
xlabel('X');
ylabel('Y');
zlabel('Z');
title('Twisted Parametric Ribbon Surface');
axis equal;
axis tight;
grid on;
view(45, 35);

print(gcf, 'twisted ribbon.png', '-dpng', '-r300');

%% 문제 16
clear; clc;

rng(3);

```

```

t = linspace(0, 60, 6000);
signal = sin(0.5 * t) + 0.4 * sin(3 * t) + 0.15 * randn(size(t));

trend = movmean(signal, 200);

x = linspace(-5, 5, 300);
y = linspace(-5, 5, 300);

[X, Y] = meshgrid(x, y);

Z = exp(-0.05 * (X.^2 + Y.^2)) .* cos(3 * X) .* sin(2 * Y);

figure;
tiledlayout(2, 2);

nexttile;
plot(t, signal);
hold on;
plot(t, trend, 'LineWidth', 1.5);
xlabel('Time');
ylabel('Signal');
title('Signal and Moving Average Trend');
legend('Signal', 'Trend');
grid on;

nexttile;
surf(X, Y, Z);
shading interp;
colorbar;
xlabel('X');
ylabel('Y');
zlabel('Z');
title('Surface Plot');

nexttile;
contourf(X, Y, Z, 50, 'LineColor', 'none');
colorbar;
xlabel('X');
ylabel('Y');
title('Filled Contour');
grid on;

sgtitle('Integrated Visualization Dashboard');

print(gcf, 'visualization_dashboard.png', '-dpng', '-r300');

```